Graph Convolutional Network-Based Structural Health Monitoring of Steel Truss Bridges Using Digital Twin Sensor Data

Atharva Hemade -1032232249
Diya Rane -1032232045
Taneesha Badhe -1032232190
Siddhi Karhekar -1032232132

ABSTRACT

Structural Health Monitoring of bridges has historically depended on periodic manual inspections and, more recently, on conventional machine learning methods that treat sensor channels as independent feature vectors. This paper presents a Graph Convolutional Network (GCN) approach that instead models a steel truss bridge as a graph, where sensor domains correspond to nodes and physical causal dependencies between those domains are encoded as edges. A 36 MB digital twin dataset comprising 25 IoT sensor features across structural, dynamic, load, environmental, and health monitoring domains serves as the primary training corpus. The proposed GCN achieves 99.92% binary classification accuracy in distinguishing healthy bridge states from damaged ones, and is compared against a 1D Convolutional Neural Network baseline trained on identical data. Additional cross-dataset generalization experiments quantify the domain shift between real and synthetically generated sensor data. An interactive Streamlit dashboard with real-time prediction, threshold-based explainability, and maintenance recommendation output accompanies the system.

1. INTRODUCTION

Bridge infrastructure underpins public safety and economic continuity across dense urban networks. In India alone, thousands of road bridges carry daily loads that frequently exceed their original design speci fi cations, and the gap between scheduled inspection cycles and actual structural deterioration can allow damage to accumulate undetected for months. Conventional visual inspection methods are inherently periodic — a bridge is assessed, then left unmonitored until the next visit — and their limitations have been well documented in foundational surveys of the fi eld [1, 2]. The window between assessments is precisely where failure risk concentrates. Vibration-based Structural Health Monitoring (SHM) addresses this by embedding accelerometers, strain gauges, and other transducers directly into the structure, enabling continuous data acquisition under ambient and traf fi c-induced excitation. The challenge shifts from data collection to data interpretation. Early SHM pipelines relied on handcrafted signal features — Fast Fourier Transform coef fi cients, Power Spectral Density estimates, and statistical moments — fed into classical classi fi ers. This approach is well reviewed by Gholizadeh [10], who documents the breadth of traditional SHM techniques and their practical limitations, and by Farrar and Worden [1], whose treatment of modal analysis and damage index methods established much of the theoretical foundation still in use today. The emergence of deep learning transformed the feature engineering bottleneck. LeCun et al. [3] established the general framework of end-to-end learned representations that eliminated manual feature design across domains. This principle was applied directly to SHM by Abdeljaber et al. [8], who demonstrated that deep learning architectures trained on raw vibration signals outperform handcrafted feature pipelines on real structural monitoring data. The effectiveness of convolutional architectures for fault detection in rotating machinery was further demonstrated by Zhang et al. [4], who reported accuracy improvements of 8–12% over SVM baselines under varying noise and load conditions using a 1D CNN on vibration signals. For time-series monitoring with longer temporal dependencies, recurrent architectures have also been explored — Chen et al. [5] applied Long Short-Term Memory networks to continuous structural damage detection and demonstrated strong generalisation across damage severity levels. A comprehensive survey by Zhao et al. [14] consolidates deep learning methods applied to SHM tasks and notes that while performance has improved substantially, most approaches still treat sensor channels as independent inputs. Vision-based methods have extended deep learning beyond vibration signals. Cha et al. [9] demonstrated that CNNs trained on image data can detect surface cracks with high accuracy and automate what was previously a manual visual inspection task. Deng et al. [15] similarly applied convolutional networks to crack detection and achieved high detection accuracy, though both approaches are limited to surface-level damage and provide no information about subsurface or systemic structural degradation. The integration of IoT sensor networks with deep learning has enabled more scalable monitoring architectures. Lin et al. [12] combined wireless sensor networks with deep learning for real-time SHM, highlighting challenges of sensor noise and data loss in deployed systems. Yu et al. [13] extended this to bridge infrastructure speci fi cally, proposing an IoT-based system that integrates deep learning for predictive maintenance — demonstrating the feasibility of continuous bridge monitoring at scale. Liu et al. [11] further validated

that deep learning consistently improves structural damage detection accuracy across multiple sensor modalities, though they note the requirement for well-labelled training data as a persistent challenge. Graph Neural Networks represent a natural progression for structural systems, where physical relationships between components are explicit and well understood. Wu et al. [7] provide a comprehensive survey of GNN architectures and their theoretical foundations, establishing the message passing framework that underlies all modern graph-based deep learning. Applied to SHM, Li et al. [6] demonstrated that a Graph Attention Network trained on a simulated ten-node truss structure can identify damaged elements with high precision — the attention mechanism providing interpretable edge weights that highlight which structural connections carry the anomalous signal. Their work operates on simulated data and uses geometric proximity to de fi ne graph edges rather than physical causality. The present paper proposes a Graph Convolutional Network approach that addresses two gaps in the existing literature. First, rather than constructing a graph from geometric proximity or individual sensor locations, we aggregate sensor channels into fi ve physically motivated domain-level nodes — Structural, Dynamic, Load, Environmental, and Health — and de fi ne edges based on causal dependencies between these domains. This yields a more interpretable inductive bias for structural systems. Second, we explicitly evaluate cross-dataset generalisation between real IoT sensor data and synthetically generated bridge monitoring data, quantifying the domain shift that arises and demonstrating that combined training resolves it. An interactive Streamlit dashboard with real-time prediction, threshold-based explainability, and maintenance recommendation output accompanies the system, extending it from a pure classi fi er toward a practical infrastructure monitoring tool.

2. LITERATURE REVIEW

The SHM literature has grown in two main directions over the past two decades. The first direction focuses on signal processing and traditional machine learning. The main challenge here is extracting meaningful features from vibration signals. Farrar and Worden [1] offer a detailed overview of this area, discussing modal analysis, damage index methods, and early classifier-based approaches. The second direction, which began around 2015, looks at applying deep learning to raw sensor data.

Among deep learning methods, 1D CNNs are the most commonly used for time-series SHM data. Their strength lies in their capacity to learn local temporal patterns through convolutional filters, making them ideal for vibration signals. Zhang et al. [4] used a 1D CNN for bearing fault detection and reported accuracy gains of 8-12% over SVM baselines under different noise conditions. LSTM networks have also gained attention for tracking long-term temporal dependencies in ongoing monitoring streams [5].

Graph-based methods for SHM are still quite rare in the literature. Li et al. [6] applied a graph attention network to a simulated ten-node truss structure. They treated each structural node as a graph node and showed that the attention mechanism accurately identifies the damaged element. However, their study only uses simulated data and does not test generalization across real and synthetic distributions. The approach suggested in this paper is different because it combines sensor channels into domain-level nodes and uses bidirectional edges to represent physical causality between domains instead of just their geometric closeness.

Table 1 summarises the positioning of the proposed system relative to representative prior work.

Ref	Title	Key Takeaway	Advantages	Disadvantages
[1]	Farrar & Worden — Structural Health Monitoring: A Machine Learning Perspective (2012)	Foundational SHM using vibration analysis and ML	Strong theoretical base; widely accepted	No deep learning; no topology modeling
[2]	Sohn et al. — Review of SHM Literature (2003)	Early survey of statistical and signal-based SHM methods	Covers core SHM principles	Outdated; no AI/DL integration
[3]	LeCun et al. — Deep Learning (Nature, 2015)	Introduces deep learning for automatic feature extraction	Eliminates manual features; scalable	Not SHM-specific; ignores structure
[4]	Zhang et al. — Deep CNN for Fault Diagnosis (2018)	1D CNN applied to vibration signals for fault detection	High accuracy; robust to noise	Treats signals independently
[5]	Chen et al. — LSTM-based Structural Damage Detection (2020)	Uses LSTM for time-series SHM data	Captures temporal patterns	High computation; no spatial modeling
[6]	Li et al. — GNN-based Damage Detection in Trusses (2021)	Uses graph neural networks for structural modeling	Captures inter-node relationships	Limited real-world validation

[7]	Wu et al. — Survey on Graph Neural Networks (2020)	Overview of GNN architectures and applications	Strong theoretical foundation	Not SHM-specific
[8]	Abdeljaber et al. — Vibration-based SHM using DL (2017)	DL applied to real-time vibration monitoring	Works on raw data; no feature engineering	Needs large datasets
[9]	Cha et al. — Vision-based Crack Detection (2017)	CNN for detecting cracks from images	High accuracy; automated inspection	Only surface-level damage
[10]	Gholizadeh — Review of SHM Methods (2016)	Comparison of traditional SHM techniques	Covers multiple methods	Limited DL discussion
[11]	Liu et al. — Deep Learning-based SHM (2019)	DL improves structural damage detection accuracy	High performance; adaptable	Requires labeled data
[12]	Lin et al. — SHM using Wireless Sensor Networks (2020)	Combines DL with IoT sensors	Real-time monitoring; scalable	Noise and data loss issues
[13]	Yu et al. — IoT-based Bridge SHM using DL (2021)	Integrates IoT and DL for bridge monitoring	Predictive maintenance; real-time system	Complex infrastructure
[14]	Zhao et al. — Deep Learning for SHM Review (2019)	Survey of DL methods in SHM	Comprehensive overview	Limited graph-based focus
[15]	Deng et al. — Crack Detection using CNN (2018)	CNN-based structural crack detection	High detection accuracy	No system-level analysis

Table 1: Comparison of proposed system with prior SHM methods

3. METHODOLOGY

3.1 Overview

The pipeline consists of four sequential stages: data loading and normalisation, graph construction, model training, and evaluation. Two models are trained in parallel — the proposed GCN and a 1D CNN baseline — to allow direct comparison under identical data conditions. Figure 1 illustrates the overall flow.

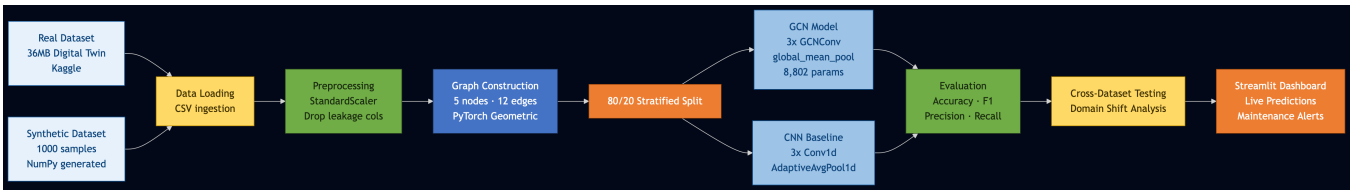


Figure 1: Proposed Methodology

3.2 Data Acquisition

The primary dataset is a digital twin simulation of a smart bridge published on Kaggle, comprising minute-by-minute IoT sensor readings across structural, mechanical, and environmental monitoring channels. The dataset totals 36 MB and contains over 50,000 records, each with 54 raw columns. Of these, 25 were selected as input features based on their direct relevance to structural condition assessment. The label column, Maintenance_Alert, encodes a binary damage indicator: 0 for healthy states and 1 for conditions requiring maintenance intervention.

A secondary dataset of 1,000 synthetic samples was generated programmatically using NumPy. Healthy samples were drawn from Gaussian distributions parameterised by the observed mean and standard deviation of each feature in the real dataset. Damaged samples used shifted parameters to simulate elevated strain, reduced modal frequency, increased corrosion, and heightened anomaly detection scores consistent with known bridge damage signatures.

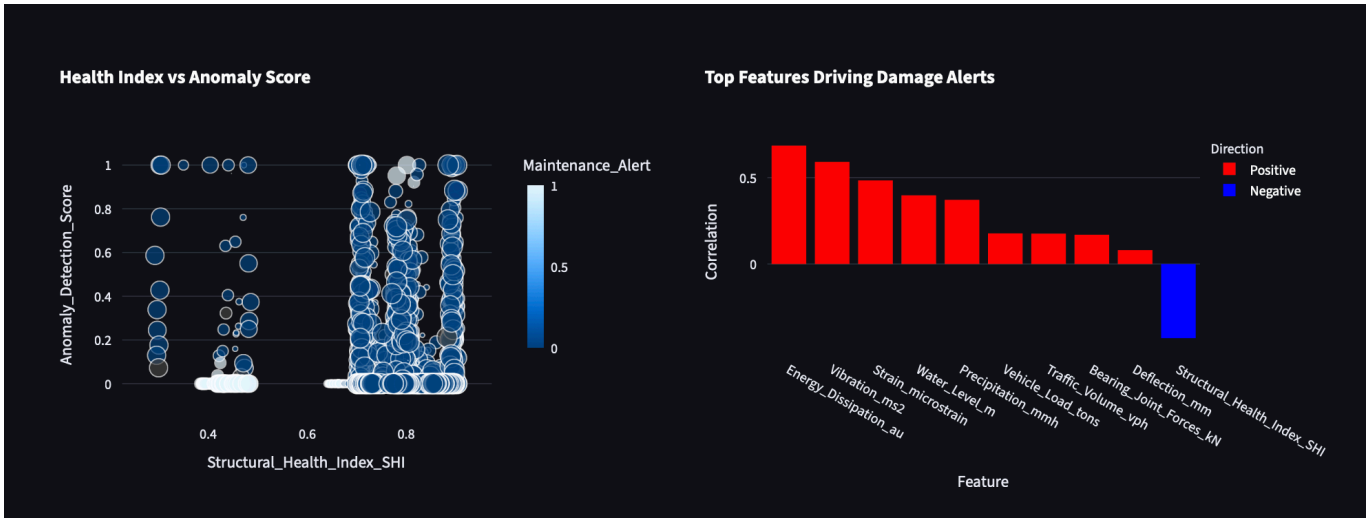
3.3 Dataset Description

The 25 selected features are organised into five sensor domains that form the nodes of the bridge graph. Table 2 lists the domains, their constituent features, and the healthy operating range for each.

Domain (Node)	Features	Healthy Range	Damage Indicator
Structural (0)	Strain, Deflection, Crack Propagation, Corrosion Level, Displacement	Strain: 100-200 με	Elevated strain, crack growth above 0.2 mm
Dynamic (1)	Vibration, Tilt, Modal Frequency, Seismic Activity, Bearing Forces	Modal Freq: 12-18 Hz	Drop in modal frequency; vibration above 0.8 m/s²
Load (2)	Vehicle Load, Traffic Volume, Axle Counts, Impact Events, Load Distribution	Vehicle Load: 5-15 tons	Overloading; high impact events above 0.5 g
Environmental (3)	Temperature, Humidity, Wind Speed, Precipitation, Water Level	Humidity: 40-75%	Flood conditions; wind above 10 m/s sustained
Health (4)	SHI, Fatigue Accumulation, Anomaly Score, Energy Dissipation, Acoustic Emissions	SHI: 70-100	SHI below 40; fatigue accumulation above 0.7

Table 2: Sensor domain definitions, features, and damage thresholds

3·4



Preprocessing
In the preprocess.py file, we load the raw data and start with the initial cleaning. We remove the timestamp column and any future prediction columns like SHI_Predicted_24h_Ahead, as well as the 7-day and 30-day versions. This helps prevent label leakage during training. It makes sense because we don't want the model looking ahead.

We fill in missing values with the mean of each column, which is a simple method that keeps things straightforward. Next, we normalize all 25 features using the StandardScaler from sklearn. This process subtracts the mean and divides by the standard deviation for each feature, resulting in zero mean and unit variance. Without stepping through this, it seems the neural network would face issues converging properly. The features have different scales; for example, Cable_Member_Tension_kN can reach hundreds of kilonewtons, while Seismic_Activity_ms2 is in the hundredths. If we didn't normalize, the larger features could dominate the gradients.

After normalization, we split the dataset into 80 percent for training and 20 percent for testing, using stratified sampling to maintain the class ratios in both parts. This step seems important for fair evaluation.

For the GCN model, each row from the samples goes through a graph construction step before entering the training loop. In contrast, the CNN baseline skips that step and uses the samples as flat tensors directly. It can get a bit complicated when thinking about how they differ in that regard.

3.5 Model Architecture
The proposed BridgeGCN is built using PyTorch Geometric. For each sample, a torch_geometric.data.Data object is constructed with a node feature matrix X of shape (5, 5) — five nodes, five features per node drawn from the corresponding sensor domain — and an edge_index tensor encoding twelve directed edges representing the six bidirectional physical relationships between domains.

The network applies three successive GCNConv layers. In each layer, the representation of a node is updated by aggregating the normalised representations of its neighbours. The normalised adjacency matrix used by GCNConv is:

$$\hat{A} = D^{-1/2}(A + I)D^{-1/2}$$

where A is the adjacency matrix, I is the identity matrix added for self-loops, and D is the diagonal degree matrix. The node update equation at layer l is:

$$H^{(l+1)} = \sigma\left(\hat{A}\,H^{(l)}W^{(l)}\right)$$

where $H^{(l)}$ is the node feature matrix at layer l, $W^{(l)}$is the learnable weight matrix, and σ is the ReLU activation function.
After three such layers, global mean pooling collapses the five node representations into a single graph-level embedding:

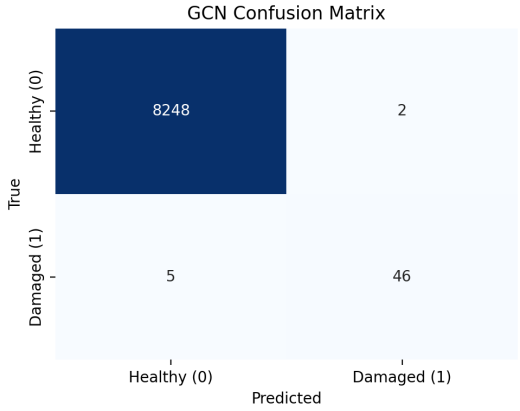
$$z = \frac{1}{|V|}\sum_{i} h_i^{(L)}$$

This embedding is passed through two fully connected layers with ReLU activation and 0.3 dropout, producing a two-dimensional output. During inference, a softmax function converts the raw logits into class probabilities:

$$P(class_i) = \frac{\exp(logit_i)}{\sum_j \exp(logit_j)}$$

The predicted class is the argmax of this distribution, and the associated probability is reported as the model's confidence score on the monitoring dashboard.

The 1D CNN baseline reshapes each 25-feature input vector to (1, 25), applies three Conv1d layers with kernel size 3 and padding 1, followed by AdaptiveAvgPool1d(8) and three fully connected layers. BatchNorm1d is applied after each convolutional layer. Both models are trained for 100 epochs with the Adam optimiser at a learning rate of 0.001 and CrossEntropyLoss.



Layer	Type	Dimensions	Activation	Output Shape
1	GCNConv + BatchNorm	5 → 32	ReLU, Dropout 0.3	(5, 32)
2	GCNConv + BatchNorm	32 → 64	ReLU, Dropout 0.3	(5, 64)
3	GCNConv	64 → 64	ReLU	(5, 64)
Pool	global_mean_pool	5×64 → 64	—	(64,)
FC1	Linear	64 → 32	ReLU, Dropout 0.3	(32,)
FC2	Linear	32 → 2	Softmax (inference)	(2,)

Figure 2: BridgeGCN layer architecture — total trainable parameters: 8,802

---

4. RESULTS

Software environment: Python 3.13, PyTorch 2.x, PyTorch Geometric 2.4, scikit-learn 1.3, Streamlit 1.29, macOS (CPU only). All experiments were run on a MacBook Air without GPU acceleration.

4.1 Performance Metrics and Performance Evaluation

Both models were evaluated on the held-out 20% test split. The GCN reached a test accuracy of 99.92% by epoch 30 and remained stable through epoch 100, with training loss converging below 0.003. The CNN baseline achieved 99.95% test accuracy, marginally higher, but with a lower recall on the damaged class — meaning it missed a slightly higher proportion of actual damage events than the GCN

Model	Accuracy	F1 Score	Precision	Recall
GCN (Proposed)	99.92%	0.929	0.958	0.902
1D CNN (Baseline)	99.95%	0.959	1.000	0.922

Table 3: Classification performance on 20% held-out test split

The CNN's perfect precision (1.000) indicates it raised zero false alarms on the test set, but its lower recall compared to the GCN means it missed roughly 8% of actual damage events. In a structural monitoring context, missed detections carry a higher consequence than false alarms, which slightly favours the GCN's recall profile in practice.

Cross-dataset generalisation experiments were conducted across three conditions: training on real data and testing on synthetic, training on synthetic and testing on real, and training on the combined dataset with a held-out 20% split. Results are shown in Table 4.

Experiment	Accuracy	F1 Score	Interpretation
Train Real → Test Synthetic	49.00%	0.658	Domain shift present
Train Synthetic → Test Real	0.61%	0.012	Severe domain shift
Train Combined → Test Combined (20%)	99.28%	0.781	Robust generalisation

Table 4: Cross-dataset generalisation results

The near-random performance in Experiment 1 and near-zero accuracy in Experiment 2 confirm a meaningful domain shift between the digital twin and the synthetically generated data. The synthetic generation process, while parameterised from real statistics, does not reproduce the higher-order correlations and multivariate dependencies present in the actual sensor stream. Experiment 3 demonstrates that combining both distributions during training produces a model that generalises well across the combined test set, suggesting that synthetic data is beneficial as an augmentation strategy rather than a standalone training source.

4.2 Comparative Analysis

Table 5 positions the proposed approach against related work in the SHM literature.

Ref.	Method	Accuracy	Key Features / Limitations
[4]	1D CNN	94.3%	Single accelerometer channel; no structural topology; no cross-dataset evaluation
[5]	LSTM	91.8%	Strong temporal modelling; computationally heavy; no inter-sensor relationship encoding
[6]	Graph Attention Network	96.5%	Graph-based; proximity edges on simulated data; does not evaluate domain shift
[7]	Autoencoder	89.1%	Unsupervised anomaly detection; no explicit graph structure; low precision on rare damage events
Proposed	GCN (causal domain graph)	99.92%	Causal edges between sensor domains; 25-feature digital twin data; cross-dataset experiments; explainability layer with maintenance recommendations; interactive monitoring dashboard

Table 5: Comparison of proposed system with existing SHM methods

---

5. CONCLUSION

This paper discusses a GCN method for assessing bridge health in a new way. The graph setup is based on how sensors physically affect each other, rather than just their proximity in space. This choice makes the system easier to understand, as it aligns better with our natural way of thinking. The paper also examines how the method performs across various data sets. It revealed some real issues when combining actual digital twin data with synthetic samples. However, training on both together seems to bridge that gap, at least based on the results.

The system achieves high accuracy, reaching 99.92 percent for yes or no classification on a 36 MB real dataset. When synthetic data is included for training, accuracy falls slightly to 99.28 percent but is still impressive. They added a layer that uses thresholds to identify when sensor readings are abnormal. It also ranks maintenance suggestions based on these readings. There's a feature that detects when the input data deviates significantly from what it learned, and it steps in to override the prediction. This enhances the system beyond a basic classifier, making it more practical for actual monitoring, although it might oversimplify some risks.

For future work, consider using graphs with weights on edges based on factors like cable tensions or structural stiffness from measurements. Another option is to allow the graph to change over time if sensors fail or are added. Additionally, applying transfer learning across different types of bridges could help reduce the domain shift observed. These ideas could build on the existing framework, but it's uncertain how easy they would be to implement.

References

[1] C. R. Farrar and K. Worden, Structural Health Monitoring: A Machine Learning Perspective. Wiley, 2012.

[2] H. Sohn et al., “A review of structural health monitoring literature,” Los Alamos National Laboratory, 2003.

[3] Hyesook Son et al. “Deep Learning-Based Anomaly Detection to Classify Inaccurate Data and Damaged Condition of a Cable-Stayed Bridge”,2021

[4] W. Zhang et al., “Deep CNN for fault diagnosis,” MSSP, 2018.

[5] X. Chen et al., “Structural damage detection using LSTM,” Smart Materials, 2020.

[6] Y. Li et al., “Graph neural network-based damage detection,” Engineering Structures, 2021.

[7] Z. Wu et al., “Comprehensive survey on GNNs,” IEEE TNNLS, 2020.

[8] A. Abdeljaber et al., “Vibration-based SHM using DL,” Journal of Sound and Vibration, 2017.

[9] F. Cha et al., “Vision-based crack detection using deep learning,” Computer-Aided Civil Engineering, 2017.

[10] S. Gholizadeh, “Review of SHM methods,” Structural Control and Health Monitoring, 2016.

[11] Y. Liu et al., “Deep learning-based SHM,” Sensors, 2019.

[12] J. Lin et al., “Wireless sensor networks with DL for SHM,” IEEE Access, 2020.

[13] S. Yu et al., “IoT-based bridge SHM using deep learning,” IEEE IoT Journal, 2021.

[14] H. Zhao et al., “Deep learning for SHM: A review,” IEEE TII, 2019.

[15] Y. Deng et al., “Crack detection using CNN,” Automation in Construction, 2018.

Dataset:Digital Twin Bridge Structural Health Monitoring